UNIT –II LOOPING, CONTROL STRUCTURES, FUNCTIONS,

RECUSRION IN R

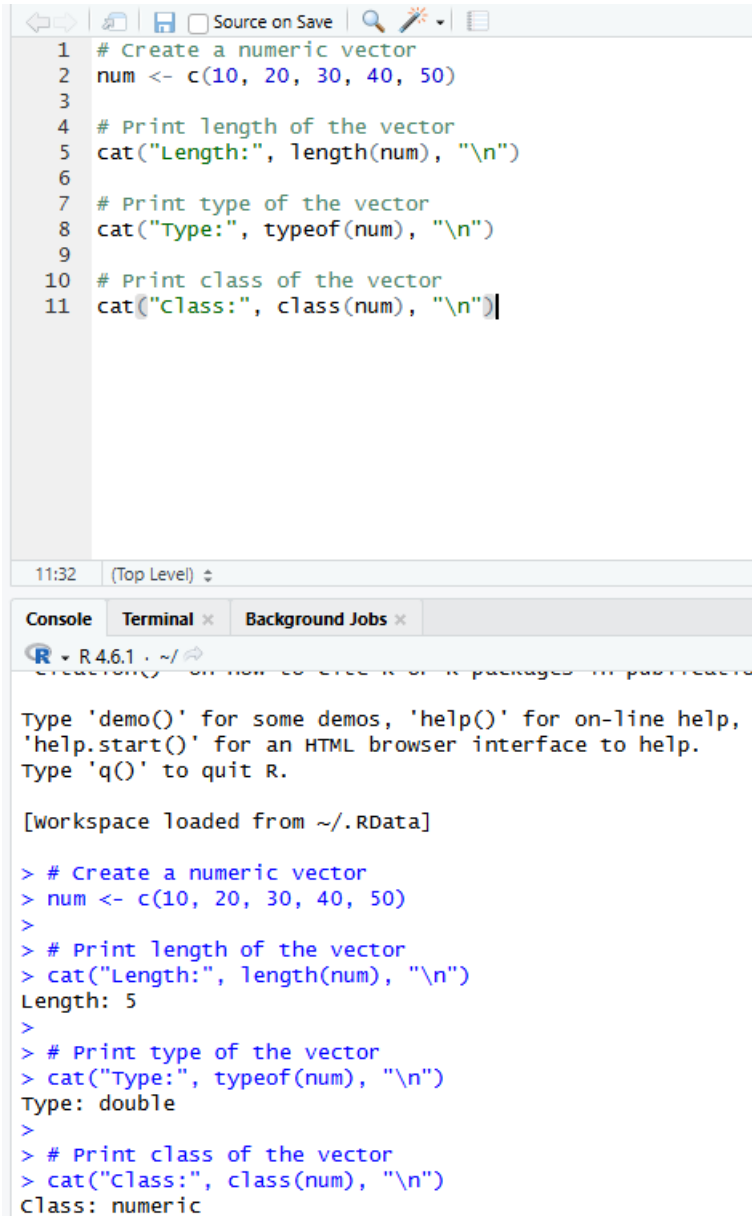
Name: Savitha R

Reg.no: 192421368

1) R Programming Structures

1. Write an R program to take a numeric vector and print its length, type (typeof), and class.

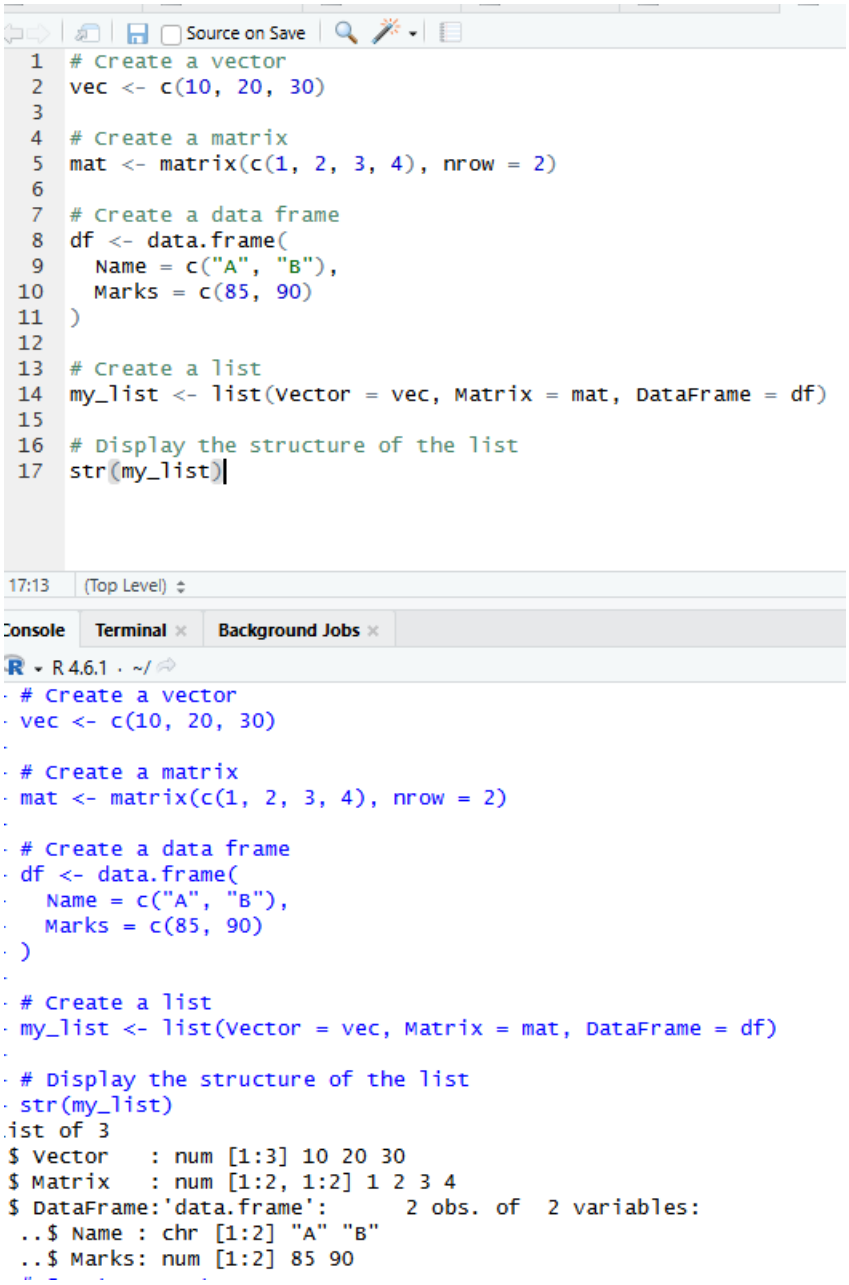
```
# Create a numeric vector
num <- c(10, 20, 30, 40, 50)

# Print length of the vector
cat("Length:", length(num), "\n")

# Print type of the vector
cat("Type:", typeof(num), "\n")

# Print class of the vector
cat("Class:", class(num), "\n")
```



2. Write an R program to create a list containing a vector, matrix, and data frame, then display the structure using str().

```
# Create a vector

vec <- c(10, 20, 30)


# Create a matrix

mat <- matrix(c(1, 2, 3, 4), nrow = 2)


# Create a data frame
```

```
df <- data.frame(
  Name = c("A", "B"),
  Marks = c(85, 90)
)

# Create a list
my_list <- list(Vector = vec, Matrix = mat, DataFrame = df)

# Display the structure of the list
str(my_list)
```

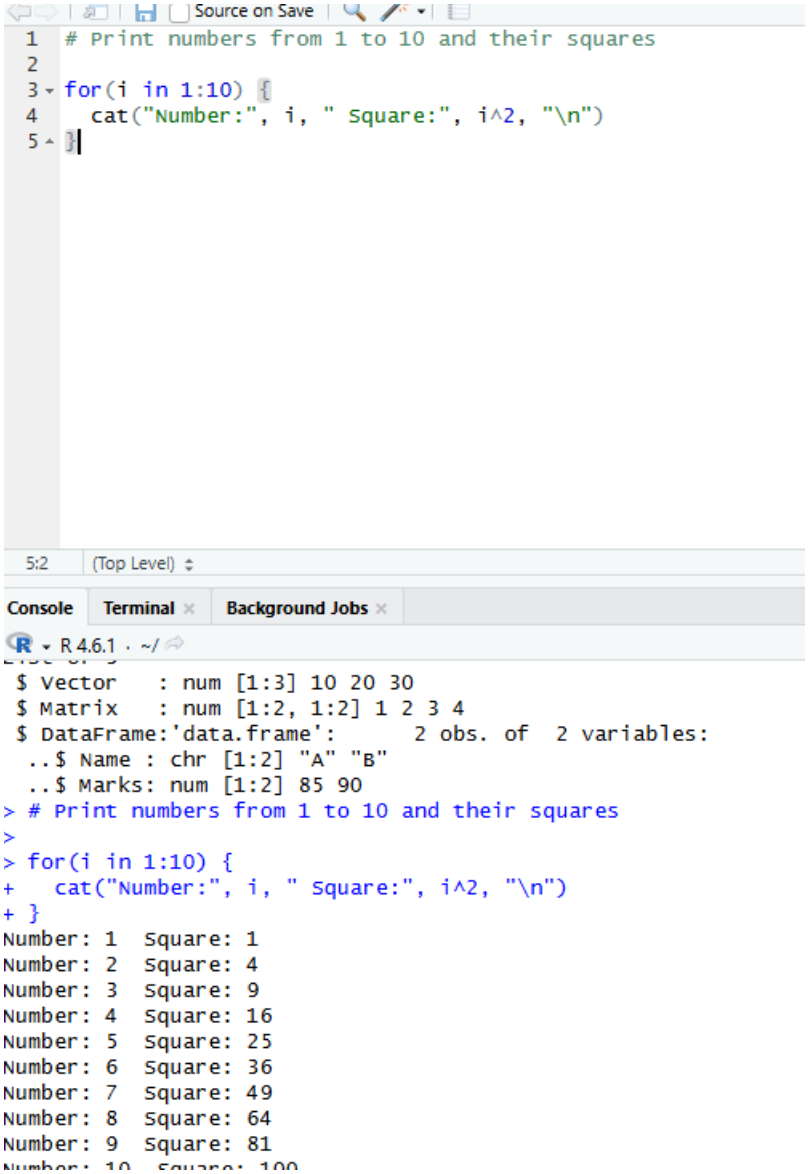


2) for() Loop

3. Write an R program using for loop to print numbers from 1 to 10 and their squares.

Print numbers from 1 to 10 and their squares

```
for(i in 1:10) {
  cat("Number:", i, " Square:", i^2, "\n")
}
```



4. Write an R program using for loop to find the sum of even numbers from 1 to 50.

Find the sum of even numbers from 1 to 50

```
sum <- 0

for(i in 1:50) {
  if(i %% 2 == 0) {
    sum <- sum + i
  }
```

}

cat("Sum of even numbers from 1 to 50 =", sum)

```
> cat("Sum of even numbers from 1 to 50 =", sum)
Sum of even numbers from 1 to 50 = 650
```

3) while() Loop

5. Write an R program using while loop to print numbers from 10 down to 1.

Print numbers from 10 to 1 using while loop

i <- 10

while(i >= 1) {

print(i)

i <- i - 1

}

```
+ }
[1] 10
[1] 9
[1] 8
[1] 7
[1] 6
[1] 5
[1] 4
[1] 3
[1] 2
[1] 1
```

6. Write an R program using while loop to find the factorial of a number n.

Find the factorial of a number using while loop

n <- 5

fact <- 1

while(n > 0) {

```
  fact <- fact * n
  n <- n - 1
}

cat("Factorial =", fact)
```

```
> # Find the factorial of a number using while loop
>
> n <- 5
> fact <- 1
>
> while(n > 0) {
+   fact <- fact * n
+   n <- n - 1
+ }
>
> cat("Factorial =", fact)
Factorial = 120
```

4) Control Statements (break, next)

7. Write an R program that prints numbers 1 to 20 but skips multiples of 3 using next.

```
> # Print numbers from 1 to 20, skipping multiples of 3
>
> for(i in 1:20) {
+   if(i %% 3 == 0) {
+     next
+   }
+   print(i)
+ }
[1] 1
[1] 2
[1] 4
[1] 5
[1] 7
[1] 8
[1] 10
[1] 11
[1] 13
[1] 14
[1] 16
[1] 17
[1] 19
[1] 20
```

8. Write an R program that scans a vector and stops when it finds a negative number using break.

```
> # Create a vector
> vec <- c(5, 10, 15, -8, 20, 25)
>
> # Scan the vector
> for(i in vec) {
+   if(i < 0) {
+     cat("Negative number found:", i, "\n")
+     break
+   }
+   print(i)
+ }
[1] 5
[1] 10
[1] 15
Negative number found: -8
```

5) Looping Over Non-Vector Sets (lists)

9. Create a list of 5 items and use a loop to print each element.

```
> # Create a list with 5 items
> my_list <- list(10, "R Programming", TRUE, 25.5, "Data Science")
>
> # Print each element using a loop
> for(item in my_list) {
+   print(item)
+ }
[1] 10
[1] "R Programming"
[1] TRUE
[1] 25.5
[1] "Data Science"
```

10. Create a list containing vectors of different lengths and use a loop to print the length of each item.

```
> # Create a list containing vectors of different lengths
> my_list <- list(
+     c(1, 2),
+     c(10, 20, 30),
+     c(5, 6, 7, 8),
+     c(100),
+     c(9, 8, 7, 6, 5)
+ )
>
> # Print the length of each vector
> for(item in my_list) {
+     cat("Length =", length(item), "\n")
+ }
Length = 2
Length = 3
Length = 4
Length = 1
Length = 5
```

6) If-Else

11. Write an R program using if-else to check whether a number is positive, negative, or zero.

```
> num <- -5
>
> if(num > 0) {
+   print("Positive")
+ } else if(num < 0) {
+   print("Negative")
+ } else {
+   print("Zero")
+ }
[1] "Negative"
```

12. Write an R program using if-else to assign grades: A (≥90), B (≥75), C (≥60), else Fail.

```
marks <- 85

if(marks >= 90) {
    print("Grade A")
} else if(marks >= 75) {
    print("Grade B")
} else if(marks >= 60) {
    print("Grade C")
} else {
    print("Fail")
}
[1] "Grade B"
```

7) Arithmetic Operators

13. Write an R program that reads two numbers and prints + , − , × , ÷ , %% , %/% results.

```
> a <- 20
> b <- 6
>
> cat("Addition =", a + b, "\n")
Addition = 26
> cat("Subtraction =", a - b, "\n")
Subtraction = 14
> cat("Multiplication =", a * b, "\n")
Multiplication = 120
> cat("Division =", a / b, "\n")
Division = 3.333333
> cat("Modulus =", a %% b, "\n")
Modulus = 2
> cat("Integer Division =", a %/% b, "\n")
Integer Division = 3
```

14. Write an R program to calculate simple interest using arithmetic operators.

```
> p <- 10000
> r <- 5
> t <- 2
>
> si <- (p * r * t) / 100
>
> cat("Simple Interest =", si)
Simple Interest = 1000
```

8) Boolean Operators & Values

15. Write an R program that checks whether a number lies between 10 and 50 using & and |.

```
>
> num <- 25
>
> if(num > 10 & num < 50) {
+     print("Number lies between 10 and 50")
+ } else {
+     print("Number does not lie between 10 and 50")
+ }
[1] "Number lies between 10 and 50"
>
> if(num < 10 | num > 50) {
+     print("Number is outside the range")
+ } else {
+     print("Number is inside the range")
+ }
[1] "Number is inside the range"
>
```

16. Write an R program to filter values in a vector that satisfy: x > 5 AND x is even.

```
> x <- c(2, 5, 6, 8, 11, 14, 3, 10)
>
> result <- x[x > 5 & x %% 2 == 0]
>
> print(result)
[1]  6  8 14 10
>
```

9) Default Values for Arguments

17. Write a function power(x, p=2) that returns x^p; call it with and without p.

```
> power <- function(x, p = 2) {
+     return(x^p)
+ }
>
> print(power(5))
[1] 25
> print(power(5, 3))
[1] 125
```

18. Write a function discount(price, rate=0.10) that returns final price after discount.

```
>
> discount <- function(price, rate = 0.10) {
+     return(price - (price * rate))
+ }
>
> print(discount(1000))
[1] 900
> print(discount(1000, 0.20))
[1] 800
```

10) Return Values (implicit vs explicit return)

19. Write a function that returns the largest of two numbers (use explicit return()).

```
>
> largest <- function(a, b) {
+     if(a > b) {
+         return(a)
+     } else {
+         return(b)
+     }
+ }
>
> print(largest(15, 25))
[1] 25
```

20. Write a function that returns the sum of a vector without using return() (implicit return).

```
>
> sumVector <- function(x) {
+     sum(x)
+ }
>
> v <- c(10, 20, 30, 40)
>
> print(sumVector(v))
[1] 100
```

11) Deciding Whether to Explicitly call return()

21. Write a function that checks if a number is prime and uses return(FALSE) early when divisible.

```
> isPrime <- function(n) {
+     if(n <= 1) {
+         return(FALSE)
+     }
+
+     for(i in 2:(n - 1)) {
+         if(n %% i == 0) {
+             return(FALSE)
+         }
+     }
+
+     return(TRUE)
+ }
>
> print(isPrime(17))
[1] TRUE
> print(isPrime(15))
[1] FALSE
```

22. Write a function that calculates average and returns "Invalid" using return() if the vector is empty.

```
>
> average <- function(x) {
+     if(length(x) == 0) {
+         return("Invalid")
+     }
+
+     mean(x)
+ }
>
> print(average(c(10, 20, 30, 40)))
[1] 25
> print(average(c()))
[1] "Invalid"
```

12) Returning Complex Objects

23. Write a function that returns mean and standard deviation as a list.

```
>
> stats <- function(x) {
+     list(
+         Mean = mean(x),
+         SD = sd(x)
+     )
+ }
>
> result <- stats(c(10, 20, 30, 40, 50))
> print(result)
$Mean
[1] 30

$SD
[1] 15.81139
```

24. Write a function that returns min, max, range as a named list.

```
>
> rangeInfo <- function(x) {
+     list(
+         Min = min(x),
+         Max = max(x),
+         Range = max(x) - min(x)
+     )
+ }
>
> result <- rangeInfo(c(10, 20, 30, 40, 50))
> print(result)
$Min
[1] 10

$Max
[1] 50

$Range
[1] 40
```

13) Functions are Objects (First-class)

25. Assign a function to a variable and call it (e.g., f <- function(x) x*2).

```
>
> f <- function(x) {
+     x * 2
+ }
>
> print(f(10))
[1] 20
```

26. Write a function that accepts another function as an argument and applies it to a vector.

```
> applyFunction <- function(v, f) {
+     f(v)
+ }
>
> x <- c(1, 2, 3, 4, 5)
>
> print(applyFunction(x, sum))
[1]  2  4  6  8 10
> print(applyFunction(x, mean))
[1] 3
>
```

14) No Pointers in R

27. Write a program showing pass-by-value behavior: modify a vector inside a function and show original remains unchanged.

```
> modifyVector <- function(v) {
+     v[1] <- 100
+     print(v)
+ }
>
> x <- c(10, 20, 30)
>
> modifyVector(x)
[1] 100  20  30
>
> print(x)
[1] 10 20 30
```

28. Write a program using <<- to modify a global variable inside a function and display the change.

```
>
> x <- 10
>
> changeValue <- function() {
+     x <<- 50
+ }
>
> changeValue()
>
> print(x)
[1] 50
```

15) Recursion

29. Write a recursive function to compute factorial(n).

```
> factorial <- function(n) {
+     if(n == 0 || n == 1) {
+         return(1)
+     } else {
+         return(n * factorial(n - 1))
+     }
+ }
>
> print(factorial(5))
[1] 120
```

30. Write a recursive function to compute Fibonacci(n).

```
> fibonacci <- function(n) {
+     if(n <= 1) {
+         return(n)
+     } else {
+         return(fibonacci(n - 1) + fibonacci(n - 2))
+     }
+ }
>
> print(fibonacci(6))
[1] 8
```